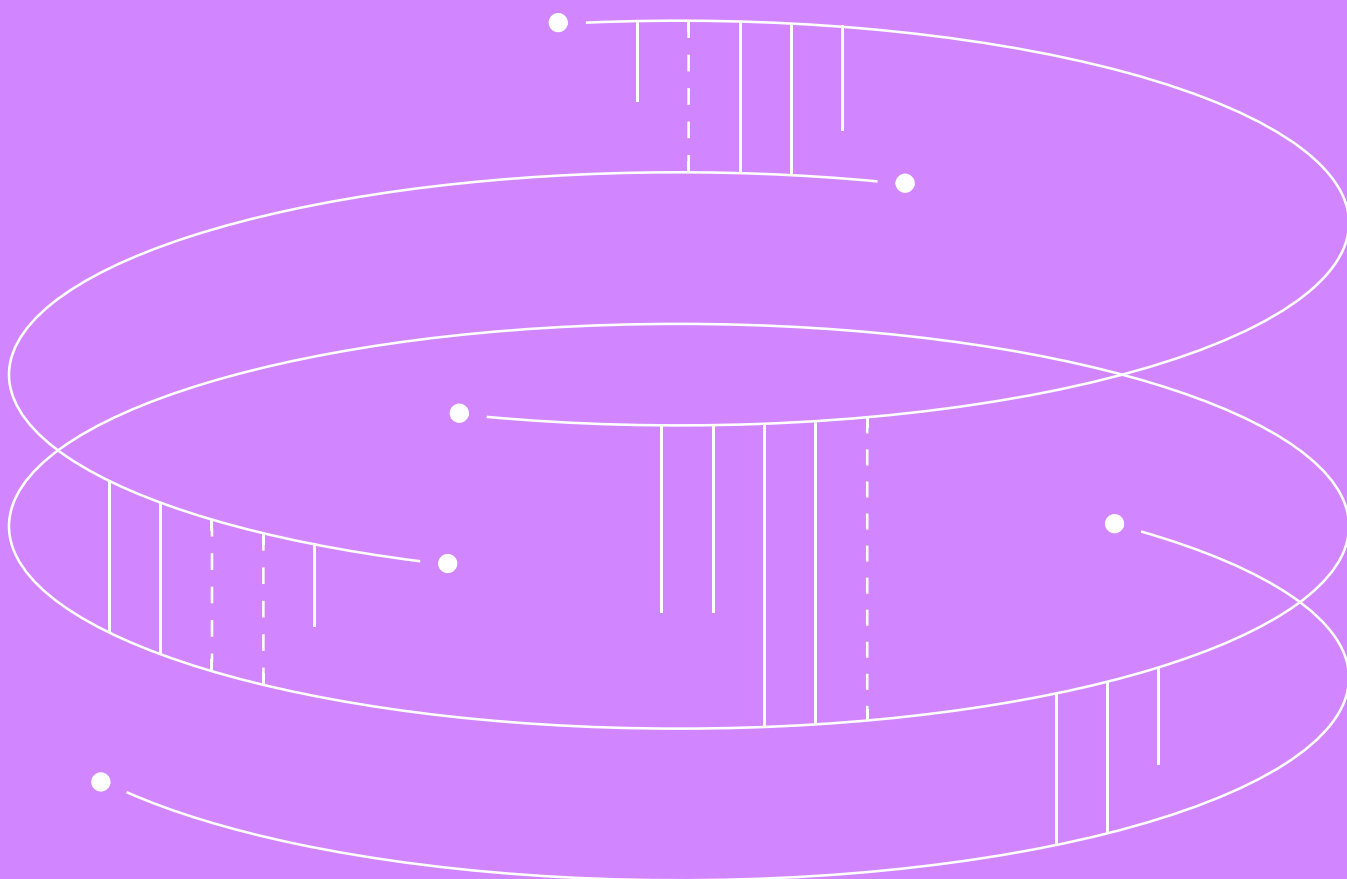


Масштабирование

РЕЗЕРВНОЕ КОПИРОВАНИЕ И РЕПЛИКАЦИЯ



SQL И БАЗЫ ДАННЫХ (ДЛЯ РАЗРАБОТЧИКОВ)

ЛОНГРИД

НЕДЕЛЯ 15

Раньше мы изучали Postgres «в вакууме»: как устроены индексы, как движок изолирует транзакции и что происходит внутри памяти. Теперь предположим: твой сервис стал популярным, пользователей всё больше, и ты видишь, что база начинает тормозить. Процессор загружен под завязку, свободная память закончилась, а диски не успевают читать и записывать данные. База становится тем самым узким местом, из-за которого страдает всё приложение.

Самое очевидное, что можно сделать в такой ситуации — это просто переехать на сервер помощнее, добавить ядер и оперативной памяти. Postgres такое решение очень любит. Исторически он заточен под работу на одной машине. У него есть общая память (shared buffers), быстрый обмен данными между процессами и один поток, который пишет журнал транзакций (WAL). На мощном «железе» всё это работает максимально эффективно.

Но у этого пути есть три проблемы, в которые ты рано или поздно упрёшься.

- Во-первых, это **деньги**. Сервер, который в два раза мощнее твоего нынешнего, будет стоить не в два, а в пять или десять раз дороже. Это просто невыгодно.
- Во-вторых, **физика**. У любого «железа» есть потолок. Ты не можешь бесконечно добавлять планки памяти или втыкать процессоры — возможности материнской платы и шины данных не безграничны.
- И в-третьих, **надёжность**. Какой бы дорогой сервер тобой ни был бы куплен, это всё равно одна коробка. Если у него сгорит блок питания, то твой сервис упадёт целиком.

Поэтому, как только такой рост становится слишком дорогим или рискованным, приходится учиться распределять нагрузку на несколько машин.

Масштабировать чтение (обычные SELECT-запросы) довольно легко — можно просто добавить ещё серверов. А вот масштабировать запись, чтобы данные не развалились и остались согласованными — это уже настоящая задача.

Обычно для этого используют схему с лидером и репликами (Leader-Follower). Мастер принимает все изменения и постоянно генерирует поток записей — WAL-лог. Этот поток по сети улетает на реплики, которые просто применяют эти изменения к своим файлам.

ОПРЕДЕЛЕНИЕ

По сути, **реплика** — это сервер, который находится в состоянии бесконечного восстановления после сбоя.

Тут начинаются компромиссы. По умолчанию в Postgres работает асинхронная репликация. Она очень быстрая: мастер записал данные у себя и сразу ответил пользователю «готово». Но появляется лаг. Внутри база измеряет его не в секундах, а в байтах — это разница между LSN (Log Sequence Number) на мастере и на реплике. Из-за этого может возникнуть странная ситуация: тобой только что создана запись, ты тут же пытаешься прочитать её с реплики, а там её ещё нет. Это называется **согласованностью в конечном счёте (Eventual Consistency)**.

Есть ещё один тонкий момент. Если на реплике идёт долгий тяжёлый запрос, а на мастере в это время обновляются те же самые строки, мастер не сможет их окончательно удалить, чтобы не «уронить» твой запрос на реплике. Для этого включают параметр `hot_standby_feedback` — реплика как бы просит мастера подождать. Но из-за этого на мастере перестает работать очистка (VACUUM), и база начинает «пухнуть». Это классический выбор: либо долгая аналитика на репликах, либо здоровье мастера.

Если данные терять нельзя совсем, включают синхронную репликацию. Мастер будет ждать подтверждения от реплик, прежде чем сказать клиенту «ок». Можно настроить кворум, например ANY 1, чтобы ждать подтверждения от любой живой реплики из списка. Это гарантирует нулевую потерю данных, но запись станет медленнее, потому что теперь мы зависим от скорости сети между серверами.

Кроме физической копии «байт в байт» есть логическая репликация. Она передаёт не изменения в файлах, а именно события: «вставлена строка», «удалена строка». Это удобно, когда нужно отправить данные в другую систему, например в Kafka, или обновить версию Postgres без остановки сервиса. Но у неё есть жесткие ограничения: она не умеет переносить изменения в структуре таблиц (DDL), не реплицирует последовательности (sequences) и большие объекты. Если ты изменишь структуру таблицы на мастере, репликация просто сломается.

Как только у нас появляются реплики, приложение больше не должно ходить напрямую в базу. Нужен посредник. Во-первых, кто-то должен понимать, куда отправлять запросы. Обычный SELECT можно отправить на реплику, а вот UPDATE нужно слать строго на мастера. Этим занимается специализированный балансировщик, например Pgpool-II, или роутинг настраивается прямо внутри приложения. Важно не путать инструменты: популярный PgBouncer — это просто пулер соединений, он не умеет разбирать SQL-запросы на части и разделять чтение и запись. Во-вторых, база данных сама себя не поднимет, если сервер с мастером внезапно сгорит. Чтобы кластер пережил падение мастера без ручного вмешательства в три часа ночи, ставится менеджер вроде Patroni. Он следит за серверами и, если мастер пропадает, аккуратно переключает роль лидера на самую свежую реплику, автоматически перенастраивая роутинг.

Бывает так, что таблица разрастается до десятков миллиардов строк. Индексы перестают помещаться в оперативную память, обслуживание вроде добавления нового индекса или выполнения VACUUM может занять часы, а обычное удаление старых записей через DELETE создаёт миллионы «мёртвых» строк и гигантскую нагрузку на диски. Здесь спасает декларативное партиционирование. Мы разбиваем огромную таблицу на куски — по диапазону дат, спискам или хешам. Приложение работает с родительской таблицей как ни в чём не бывало, а база сама перенаправляет данные в нужные партиции.

Это даёт два больших плюса.

1. Планировщик отбрасывает лишние партиции ещё до выполнения запроса — если ты ищешь события за январь, база будет читать только один физический январский файл.
2. И самое главное — ты получаешь бесплатный жизненный цикл данных. Ты перестаёшь удалять их через тяжёлый DELETE. Старые данные можно просто отбрасывать мгновенной операцией DROP TABLE. Правда, в высоконагруженной среде нельзя делать прямой DROP. Лучше сначала отцепить кусок командой DETACH PARTITION CONCURRENTLY, чтобы не повесить жёсткие блокировки на родительскую таблицу, а уже потом его удалять.

В базовом сценарии партиции используют одни и те же ресурсы процессора и памяти на одном сервере. Однако мы можем масштабировать дисковый ввод-вывод. Для этого разные партиции явно размещаются в разных табличных пространствах (tablespaces), которые физически смонтированы на разные жёсткие диски или LUN дискового массива. Это позволяет кратно увеличить пропускную способность записи в рамках одного сервера.

Попытка настраивать базу без цифр и метрик — это, честно говоря, чистое гадание. В самом Postgres есть отличный инструмент для этого — **расширение pg_stat_statements**. Оно собирает статистику по всем запросам, которые проходят через систему. Тут важно не попасть в ловушку: многие смотрят только на среднее время выполнения запроса (`mean_exec_time`). Но запрос, который идёт секунду, но вызывается всего десять раз в день — это не страшно. Настоящая проблема — это какой-нибудь лёгкий селект по ID, который отрабатывает мгновенно, но вызывается по 50 тысяч раз в секунду. Именно он незаметно съедает все ресурсы. Поэтому всегда полезно сортировать статистику по суммарному времени (`total_exec_time`) — это сразу покажет, на что на самом деле уходят силы процессора.

Но что делать, когда интенсивность именно записи превышает возможности даже самого лучшего дискового массива? Это тот самый предел архитектуры с одним мастером. Можно, конечно, попробовать «ручной шардинг» — когда приложение само раскидывает данные по разным независимым базам. Но тогда ты сразу теряешь возможность делать JOIN между этими кусками данных, лишаешься нормальных транзакций, а любая попытка добавить новый сервер превращается в головную боль.

Для таких случаев в экосистеме Postgres есть нативное решение — **расширение Citus**, которое превращает его в распределённую базу данных. Если правильно выбрать ключ, по которому данные будут делиться (`sharding key`), Citus будет сам маршрутизировать запросы по узлам, делать быстрые локальные JOIN и обеспечивать строгую надёжность через протокол двухфазного коммита.

И только когда логика твоего приложения совсем не ложится на реляционную модель или когда шардировать реляционную базу становится слишком больно и дорого, мы смотрим в сторону NoSQL. Там мы получаем децентрализованные системы с несколькими мастерами (Multi-Master), где нет единой точки записи, а шардирование работает просто «из коробки». Но это уже совершенно другая архитектура.

Подведём итоги

- Если хочешь, чтобы кластер сам переживал падения серверов и автоматически переключался на резерв (High Availability), используй инструменты вроде Patroni.
- Чётко понимай разницу между обычным пулингом соединений (PgBouncer) и маршрутизацией SQL-запросов (Pgpool-II или на уровне приложения). Это разные вещи.
- Смело используй реплики, чтобы разгрузить мастера от тяжёлого чтения.
- При поиске «узких мест» всегда смотри в `pg_stat_statements` и ориентируйся на суммарную нагрузку от запросов (`total_exec_time`), а не на среднее время выполнения одного запроса.
- И если понимаешь, что таблица вырастет до гигантских размеров, закладывай партиционирование и распределение по разным дискам (`tablespaces`) с самого начала.